# Lab 03.1 — Linear Regression with the Kaggle House Prices Dataset

**Course:** Data Analysis with Python — Data Science Linear Regression Lab **Dataset:** *House Prices – Advanced Regression Techniques* (Kaggle), `train.csv` — 1,460 homes sold in Ames, Iowa (2006–2010) **Target variable:** `SalePrice` (USD)

---

## Contents


| Section | Content |
|---|---|
| **Step 1** | Data preparation: load, inspect, missing-value audit and imputation strategy |
| **Step 2** | Exploratory data analysis: scatter plots and Pearson/Spearman correlations |
| **Step 3** | Build the linear regression model (80/20 split), intercept and coefficients |
| **Step 4** | Evaluate: MSE, RMSE, MAE, $R^2$, actual-vs-predicted and residual diagnostics |
| **Step 5** | Extension: more features, standardisation, interaction terms, multicollinearity (VIF) |
| **Report** | Strengths, limitations and consolidated answers to every discussion question |


Every **Discussion Questions** block from the lab sheet is answered in a markdown cell immediately after the code that produces the evidence for it.

## Step 0 — Setup

Imports and a single shared plotting style, so every figure in the report is consistent: one accessible colour per series, recessive grid and axes, thin marks. Figures are also written to `figures/` as PNGs for reuse in slides.

In [1]:

```
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
from scipy import stats

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore", category=FutureWarning)
RANDOM_STATE = 42          # fixed so every number in this report is
```

```
reproducible
pd.set_option("display.width", 160)
pd.set_option("display.max_columns", 60)

FIGDIR = Path("figures"); FIGDIR.mkdir(exist_ok=True)

# --- Plot style: validated accessible palette, recessive chrome ----------
--
INK, INK_SOFT, GRID = "#0b0b0b", "#52514e", "#e5e5e2"
BLUE, ORANGE, AQUA  = "#2a78d6", "#eb6834", "#1baf7a"   # categorical slots
1–3
mpl.rcParams.update({
    "figure.dpi": 110, "savefig.dpi": 150, "savefig.bbox": "tight",
    "figure.facecolor": "white", "axes.facecolor": "white",
    "axes.edgecolor": GRID, "axes.linewidth": 0.8,
    "axes.labelcolor": INK_SOFT, "axes.titlecolor": INK,
    "axes.titlesize": 11, "axes.titleweight": "bold", "axes.titlelocation":
"left",
    "axes.titlepad": 10, "axes.labelsize": 9.5,
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.grid": True, "grid.color": GRID, "grid.linewidth": 0.7,
"axes.axisbelow": True,
    "xtick.color": INK_SOFT, "ytick.color": INK_SOFT,
    "xtick.labelsize": 9, "ytick.labelsize": 9,
    "xtick.direction": "out", "ytick.direction": "out",
    "legend.frameon": False, "legend.fontsize": 9,
    "lines.linewidth": 2.0, "font.size": 10,
})

def thousands(x, _pos=None):
    # Axis formatter: 150000 -> "150k"
    return f"{x/1_000:,.0f}k" if abs(x) >= 1_000 else f"{x:,.0f}"

MONEY = mpl.ticker.FuncFormatter(thousands)

def save(fig, name):
    fig.savefig(FIGDIR / f"{name}.png")
    return fig

print("pandas", pd.__version__, "| numpy", np.__version__)
import sklearn; print("scikit-learn", sklearn.__version__)
```

```
pandas 3.0.3 | numpy 2.4.4
scikit-learn 1.9.1
```

---

# Step 1 — Data Preparation

**Tasks:** load `train.csv`, display basic information (rows/columns, data types), and check for missing data.

In [2]:

```
CSV = Path("house-prices-advanced-regression-techniques/train.csv")
df_raw = pd.read_csv(CSV)
```

```
print(f"Rows: {df_raw.shape[0]:,}   Columns: {df_raw.shape[1]}")
print(f"Target: SalePrice  |  Predictor columns: {df_raw.shape[1] - 2}
(excluding Id and SalePrice)")
print("\nColumn data types:")
print(df_raw.dtypes.value_counts().to_string())
print(f"\nNumeric columns: {df_raw.select_dtypes('number').shape[1]}"
      f"   |   Categorical (text) columns:
{df_raw.select_dtypes(exclude='number').shape[1]}")
print(f"\nDuplicate rows: {df_raw.duplicated().sum()}   |   Duplicate Ids:
{df_raw['Id'].duplicated().sum()}")
df_raw.head()
```

```
Rows: 1,460   Columns: 81
Target: SalePrice  |  Predictor columns: 79  (excluding Id and SalePrice)

Column data types:
str        43
int64      35
float64     3

Numeric columns: 38   |   Categorical (text) columns: 43

Duplicate rows: 0   |   Duplicate Ids: 0
```

Out[2]:

| | Id | MSSubClass | MSZoning | LotFrontage | LotArea | Street | Alley | LotShape | LandCon |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 60 | RL | 65.0 | 8450 | Pave | NaN | Reg | |
| **1** | 2 | 20 | RL | 80.0 | 9600 | Pave | NaN | Reg | |
| **2** | 3 | 60 | RL | 68.0 | 11250 | Pave | NaN | IR1 | |
| **3** | 4 | 70 | RL | 60.0 | 9550 | Pave | NaN | IR1 | |
| **4** | 5 | 60 | RL | 84.0 | 14260 | Pave | NaN | IR1 | |

5 rows × 81 columns

In [3]:

```
# Target variable at a glance
sp = df_raw["SalePrice"]
print("SalePrice summary (USD)")
print(sp.describe().apply(lambda v: f"{v:,.0f}").to_string())
print(f"\nSkewness: {sp.skew():.3f}  (right-skewed)   |   after log:
{np.log(sp).skew():.3f}")
```

```
SalePrice summary (USD)
count      1,460
mean     180,921
std       79,443
min       34,900
25%      129,975
50%      163,000
75%      214,000
max      755,000

Skewness: 1.883  (right-skewed)   |   after log: 0.121
```

In [4]:

```python
# ---- Missing-value audit ------------------------------------------------
-
na = df_raw.isna().sum()
na = na[na > 0].sort_values(ascending=False)
missing = pd.DataFrame({"n_missing": na, "pct_missing": (na / len(df_raw) *
100).round(2)})

print(f"Columns with at least one missing value: {len(missing)} of
{df_raw.shape[1]}")
print(f"Cells missing overall: {df_raw.isna().sum().sum():,} "
      f"({df_raw.isna().sum().sum() / df_raw.size * 100:.2f}% of the
table)")
missing
```

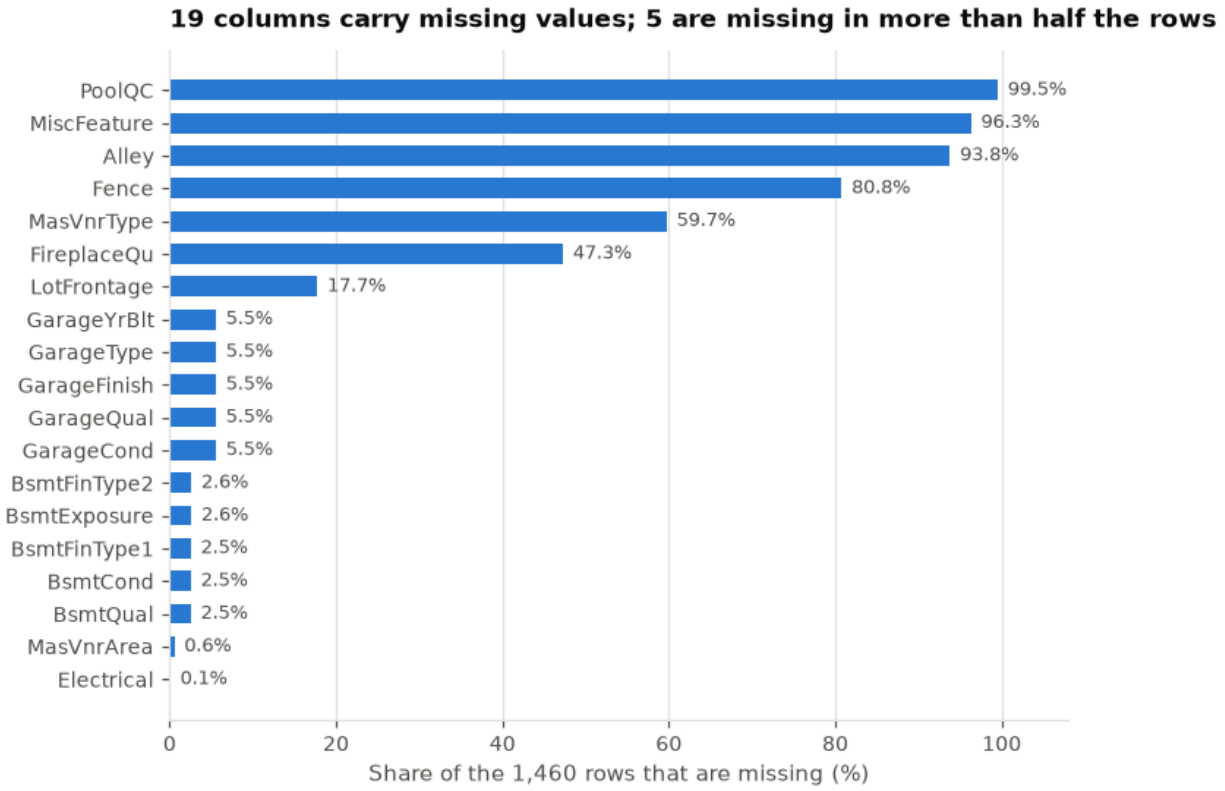
```
Columns with at least one missing value: 19 of 81
Cells missing overall: 7,829 (6.62% of the table)
```

Out[4]:

| | n_missing | pct_missing |
|---|---|---|
| **PoolQC** | 1453 | 99.52 |
| **MiscFeature** | 1406 | 96.30 |
| **Alley** | 1369 | 93.77 |
| **Fence** | 1179 | 80.75 |
| **MasVnrType** | 872 | 59.73 |
| **FireplaceQu** | 690 | 47.26 |
| **LotFrontage** | 259 | 17.74 |
| **GarageType** | 81 | 5.55 |
| **GarageYrBlt** | 81 | 5.55 |
| **GarageFinish** | 81 | 5.55 |
| **GarageQual** | 81 | 5.55 |
| **GarageCond** | 81 | 5.55 |
| **BsmtFinType2** | 38 | 2.60 |
| **BsmtExposure** | 38 | 2.60 |
| **BsmtFinType1** | 37 | 2.53 |
| **BsmtCond** | 37 | 2.53 |
| **BsmtQual** | 37 | 2.53 |
| **MasVnrArea** | 8 | 0.55 |
| **Electrical** | 1 | 0.07 |

In [5]:

```
fig, ax = plt.subplots(figsize=(7.2, 5.4))
order = missing.sort_values("pct_missing")
ax.barh(order.index, order["pct_missing"], color=BLUE, height=0.62)
for name, pct in order["pct_missing"].items():
    ax.text(pct + 1.2, name, f"{pct:.1f}%", va="center", fontsize=8.5,
color=INK_SOFT)
ax.set_xlim(0, 108)
ax.set_xlabel("Share of the 1,460 rows that are missing (%)")
ax.set_title("19 columns carry missing values; 5 are missing in more than
half the rows")
ax.grid(axis="y", visible=False)
save(fig, "01_missing_values"); plt.show()
```




## Missing-data strategy

Reading `data_description.txt` is essential here: for most of these columns `NA` is **not** an unrecorded value — it is a documented category meaning *the feature does not exist* ( `PoolQC` = "NA: No Pool", `Alley` = "No alley access", `BsmtQual` = "No Basement", `GarageType` = "No Garage", `FireplaceQu` = "No Fireplace"). Deleting those rows would throw away 1,453 houses simply for not having a pool. The strategy therefore splits into four rules:

| Rule | Columns | Action | Reason |
|---|---|---|---|
| 1. Structural `NA` → explicit category | `PoolQC` , `MiscFeature` , `Alley` , `Fence` , `FireplaceQu` , `Garage*` , `Bsmt*` , `MasVnrType` | fill with the string `"None"` | `NA` means "feature absent" — that is information, not a gap |
| 2. Numeric counterpart of an absent feature | `GarageYrBlt` , `MasVnrArea` | fill with `0` | no garage ⇒ no garage year; no veneer ⇒ 0 ft² |
| 3. Genuinely unknown numeric | `LotFrontage` (17.7%) | **median within `Neighborhood`** | frontage is driven by the street layout, so neighbourhood peers are the best estimate; the median resists the right skew |
| 4. Genuinely unknown categorical | `Electrical` (1 row) | mode of the column | a single row; the mode ( `SBrkr` ) is the safe fill |

**Why not just drop?** Dropping rows with any `NA` leaves **0 usable rows** (every house is missing at least one of `PoolQC` / `Alley` / `Fence`); dropping the 19 columns would discard useful predictors such as `LotFrontage` and `FireplaceQu`. Imputation preserves all 1,460 observations.

One point worth stressing for the modelling that follows: **the four features used in Steps 2–4 (`GrLivArea`, `OverallQual`, `YearBuilt`, `GarageArea`) and the target `SalePrice` have no missing values at all**, so the regression results do not depend on any imputation choice. The cleaning below makes the full table usable and keeps the pipeline honest for the extension in Step 5.

In [6]:

```
df = df_raw.copy()

# Rule 1 — "NA" is a real category meaning the feature is absent
none_cols = ["PoolQC", "MiscFeature", "Alley", "Fence", "FireplaceQu",
"MasVnrType",
             "GarageType", "GarageFinish", "GarageQual", "GarageCond",
             "BsmtQual", "BsmtCond", "BsmtExposure", "BsmtFinType1",
"BsmtFinType2"]
df[none_cols] = df[none_cols].fillna("None")

# Rule 2 — numeric counterpart of an absent feature
df[["GarageYrBlt", "MasVnrArea"]] = df[["GarageYrBlt",
"MasVnrArea"]].fillna(0)

# Rule 3 — LotFrontage: median within the neighbourhood
df["LotFrontage"] = df.groupby("Neighborhood")["LotFrontage"].transform(
    lambda s: s.fillna(s.median()))
df["LotFrontage"] = df["LotFrontage"].fillna(df["LotFrontage"].median())  #
safety net

# Rule 4 — single unknown categorical
df["Electrical"] = df["Electrical"].fillna(df["Electrical"].mode()[0])

print(f"Missing values remaining: {df.isna().sum().sum()}")
print(f"Rows kept: {len(df):,} of {len(df_raw):,}
({len(df)/len(df_raw)*100:.0f}% — nothing dropped)")
print("\nSanity check — rows that would survive a naive dropna():",
      len(df_raw.dropna()))
```

```
Missing values remaining: 0
Rows kept: 1,460 of 1,460  (100% — nothing dropped)

Sanity check — rows that would survive a naive dropna(): 0
```

## Discussion questions — Step 1

**Q1. How many columns have missing values? 19 of the 81 columns** (23.5%), accounting for 7,829 missing cells — 6.62% of all cells in the table. Five columns are missing in more than half their rows: `PoolQC` (99.5%), `MiscFeature` (96.3%), `Alley` (93.8%), `Fence` (80.8%) and `MasVnrType` (59.7%). At the other end, `Electrical` is

missing in a single row. The five `Garage*` columns are missing in exactly the same 81 rows, and the five `Bsmt*` columns in the same ~37 rows — a strong hint that the missingness is structural rather than random.

**Q2. What strategy did you choose to handle missing data, and why?** A **category-aware imputation** rather than deletion, in the four rules tabulated above: `"None"` for absent-feature categoricals, `0` for their numeric counterparts, neighbourhood median for `LotFrontage`, column mode for `Electrical`. The reasoning:

1. **The missingness is informative, not random.** `data_description.txt` documents `NA` as a *level* ("No Pool", "No Garage"). Treating it as a gap destroys a real signal: having no basement is a genuine, price-relevant fact.
2. **Deletion is not available.** `dropna()` on rows returns **0 rows**; dropping the 19 columns loses real predictors. Imputation keeps all 1,460 houses.
3. **Grouped median for `LotFrontage`.** It is the only sizeable column where `NA` really means "unknown". Lot frontage is determined by street and block layout, so the `Neighborhood` median is a much better estimate than a global mean, and the median is robust to the long right tail of lot sizes.
4. **No leakage.** Every fill uses predictor information only — never `SalePrice`.

---

# Step 2 — Exploratory Data Analysis

**Tasks:** select 3–4 numerical features likely related to `SalePrice`, plot each against the target, and compute the Pearson correlations.

## The four features and why they were chosen

| Feature | Meaning (`data_description.txt`) | Why it should drive price |
|---|---|---|
| `GrLivArea` | Above-grade living area, ft² | Size is the headline number in any listing; price per ft² is how the market quotes value |
| `OverallQual` | Overall material & finish quality, ordinal 1–10 | Captures finish level and condition, which size alone cannot |
| `YearBuilt` | Original construction year | Proxy for age, layout fashion, insulation, wiring and remaining service life |
| `GarageArea` | Garage size, ft² | A priced amenity in Iowa's climate and a proxy for the overall scale of the property |

`OverallQual` is an *ordinal* integer, not a continuous measurement. Treating it as numeric — as the lab asks — assumes the steps 1→2 and 9→10 are worth the same amount of money. That assumption is imperfect (the real pattern is convex) and is examined with an interaction term in Step 5.

In [7]:

```
FEATURES = ["GrLivArea", "OverallQual", "YearBuilt", "GarageArea"]
TARGET = "SalePrice"

print("Missing values in the modelling columns:",
      df[FEATURES + [TARGET]].isna().sum().sum(), "\n")
df[FEATURES + [TARGET]].describe().round(2)
```

```
Missing values in the modelling columns: 0
```

Out[7]:

| | GrLivArea | OverallQual | YearBuilt | GarageArea | SalePrice |
|---|---|---|---|---|---|
| **count** | 1460.00 | 1460.00 | 1460.00 | 1460.00 | 1460.0 |
| **mean** | 1515.46 | 6.10 | 1971.27 | 472.98 | 180921.2 |
| **std** | 525.48 | 1.38 | 30.20 | 213.80 | 79442.5 |
| **min** | 334.00 | 1.00 | 1872.00 | 0.00 | 34900.0 |
| **25%** | 1129.50 | 5.00 | 1954.00 | 334.50 | 129975.0 |
| **50%** | 1464.00 | 6.00 | 1973.00 | 480.00 | 163000.0 |
| **75%** | 1776.75 | 7.00 | 2000.00 | 576.00 | 214000.0 |
| **max** | 5642.00 | 10.00 | 2010.00 | 1418.00 | 755000.0 |

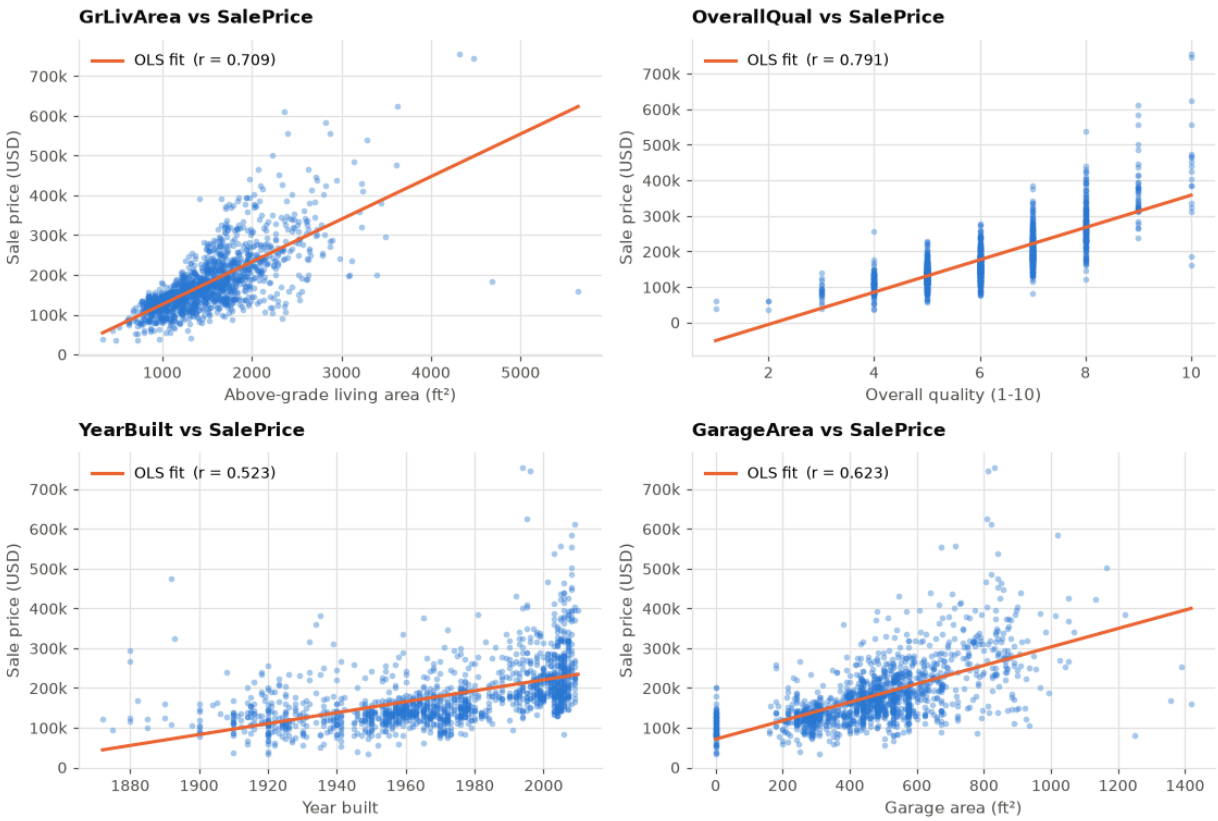
In [8]:

```
# ---- Scatter plots: each feature against SalePrice ----------------------
fig, axes = plt.subplots(2, 2, figsize=(10.4, 7.6))
LABELS = {"GrLivArea": "Above-grade living area (ft²)",
          "OverallQual": "Overall quality (1–10)",
          "YearBuilt": "Year built",
          "GarageArea": "Garage area (ft²)"}

for ax, feat in zip(axes.ravel(), FEATURES):
    x, y = df[feat], df[TARGET]
    ax.scatter(x, y, s=13, color=BLUE, alpha=0.40, linewidths=0, zorder=3)
    # least-squares trend line for the single variable
    b1, b0 = np.polyfit(x, y, 1)
    xs = np.linspace(x.min(), x.max(), 100)
    ax.plot(xs, b0 + b1 * xs, color=ORANGE, lw=2, zorder=4,
            label=f"OLS fit  (r = {stats.pearsonr(x, y)[0]:.3f})")
    ax.set_xlabel(LABELS[feat]); ax.set_ylabel("Sale price (USD)")
    ax.yaxis.set_major_formatter(MONEY)
    ax.set_title(f"{feat} vs SalePrice")
    ax.legend(loc="upper left")

fig.suptitle("All four features rise with price; the spread widens as price
rises",
             x=0.005, ha="left", fontsize=12.5, fontweight="bold",
color=INK)
fig.tight_layout(rect=[0, 0, 1, 0.97])
save(fig, "02_scatter_features"); plt.show()
```

**All four features rise with price; the spread widens as price rises**




In [9]:

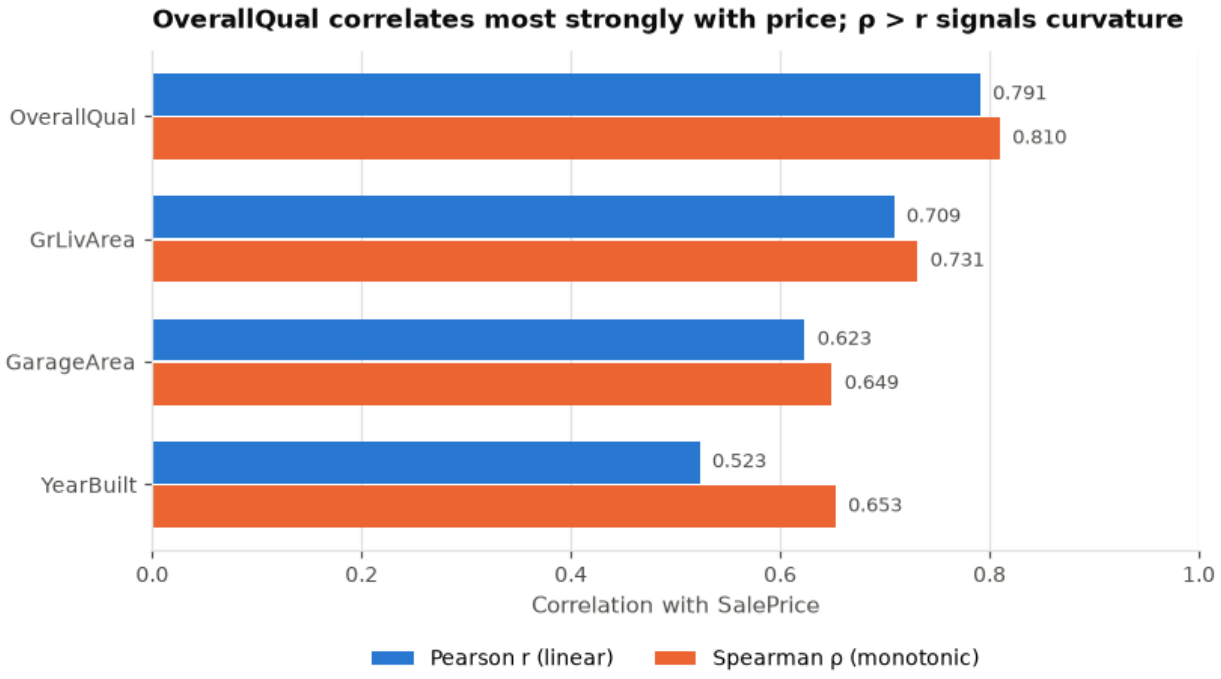
```
# ---- Pearson (linear) and Spearman (monotonic) correlations ---------------
rows = []
for feat in FEATURES:
    r, p   = stats.pearsonr(df[feat], df[TARGET])
    rs, ps = stats.spearmanr(df[feat], df[TARGET])
    rows.append({"feature": feat, "pearson_r": round(r, 4), "p_value": f"
{p:.2e}",
                 "r_squared_alone": round(r**2, 4), "spearman_rho":
round(rs, 4),
                 "gap_rho_minus_r": round(rs - r, 4),
                 "direction": "positive" if r > 0 else "negative"})
corr_tbl = pd.DataFrame(rows).sort_values("pearson_r",
ascending=False).reset_index(drop=True)
corr_tbl
```

Out[9]:

| | feature | pearson_r | p_value | r_squared_alone | spearman_rho | gap_rho_minus_r |
|---|---|---|---|---|---|---|
| **0** | OverallQual | 0.7910 | 2.19e-313 | 0.6257 | 0.8098 | 0.0188 |
| **1** | GrLivArea | 0.7086 | 4.52e-223 | 0.5021 | 0.7313 | 0.0227 |
| **2** | GarageArea | 0.6234 | 5.27e-158 | 0.3887 | 0.6494 | 0.0259 |
| **3** | YearBuilt | 0.5229 | 2.99e-103 | 0.2734 | 0.6527 | 0.1298 |

In [10]:

```
fig, ax = plt.subplots(figsize=(7.6, 4.4))
t = corr_tbl.sort_values("pearson_r")
ypos = np.arange(len(t)); h = 0.34
ax.barh(ypos + h/2 + 0.01, t["pearson_r"],   height=h, color=BLUE,
label="Pearson r (linear)")
ax.barh(ypos - h/2 - 0.01, t["spearman_rho"], height=h, color=ORANGE,
label="Spearman ρ (monotonic)")
for i, (pr, sr) in enumerate(zip(t["pearson_r"], t["spearman_rho"])):
    ax.text(pr + 0.012, i + h/2 + 0.01, f"{pr:.3f}", va="center",
fontsize=8.5, color=INK_SOFT)
    ax.text(sr + 0.012, i - h/2 - 0.01, f"{sr:.3f}", va="center",
fontsize=8.5, color=INK_SOFT)
ax.set_yticks(ypos, t["feature"])
ax.set_xlim(0, 1.0); ax.set_xlabel("Correlation with SalePrice")
ax.set_title("OverallQual correlates most strongly with price; ρ > r signals
curvature")
ax.grid(axis="y", visible=False)
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.16), ncol=2)   # below
the axis, clear of the bars
fig.tight_layout()
save(fig, "03_correlations"); plt.show()
```




In [11]:

```
# ---- Is the relationship linear? Three diagnostics ----------------------
# (a) mean price per quality level - the ordinal assumption
qual = df.groupby("OverallQual")[TARGET].agg(["count", "mean"]).round(0)
qual["step_up_vs_previous"] = qual["mean"].diff().round(0)
print("Mean SalePrice by OverallQual level\n", qual.to_string(), "\n")

# (b) spread of price as size grows - constant variance?
bins = pd.cut(df["GrLivArea"], [0, 1000, 1400, 1800, 2200, 6000])
print("Dispersion of SalePrice by GrLivArea band\n",
      df.groupby(bins, observed=True)[TARGET].agg(n="size", mean="mean",
```

```
std="std").round(0).to_string())

# (c) the four extreme-size houses
print("\nHouses with GrLivArea > 4000 ft² (classic outliers in this
dataset):")
print(df.loc[df["GrLivArea"] > 4000,
             ["GrLivArea", "OverallQual", "SaleCondition",
TARGET]].to_string())
```

```
Mean SalePrice by OverallQual level
              count      mean  step_up_vs_previous
OverallQual
1                 2   50150.0                  NaN
2                 3   51770.0               1620.0
3                20   87474.0              35704.0
4               116  108421.0              20947.0
5               397  133523.0              25102.0
6               374  161603.0              28080.0
7               319  207716.0              46113.0
8               168  274736.0              67020.0
9                43  367513.0              92777.0
10               18  438588.0              71075.0

Dispersion of SalePrice by GrLivArea band
                 n      mean       std
GrLivArea
(0, 1000]      231  112985.0   26151.0
(1000, 1400]   432  142960.0   33630.0
(1400, 1800]   457  191291.0   54022.0
(1800, 2200]   204  236185.0   70289.0
(2200, 6000]   136  299155.0  120721.0

Houses with GrLivArea > 4000 ft² (classic outliers in this dataset):
      GrLivArea  OverallQual SaleCondition  SalePrice
523        4676           10       Partial     184750
691        4316           10        Normal     755000
1182       4476           10       Abnorml     745000
1298       5642           10       Partial     160000
```

In [12]:

```
fig, axes = plt.subplots(1, 3, figsize=(12.6, 3.9))

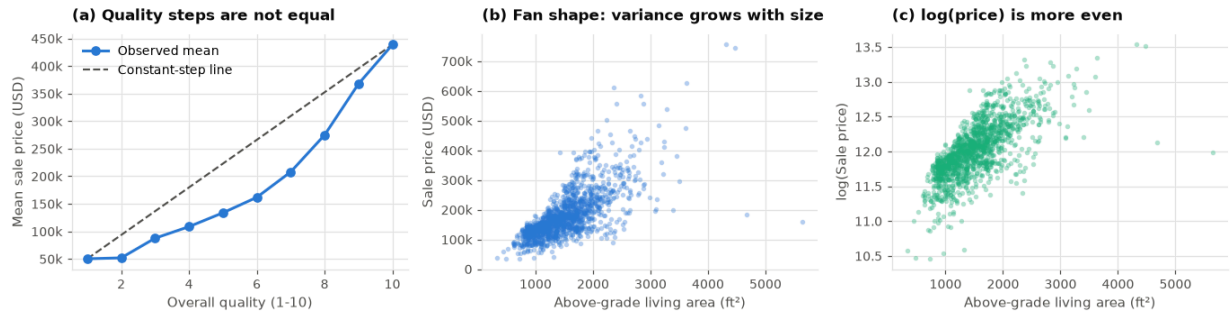
# panel 1 — curvature in the quality ladder
ax = axes[0]
ax.plot(qual.index, qual["mean"], color=BLUE, marker="o", ms=6, zorder=3,
label="Observed mean")
lo, hi = qual.index.min(), qual.index.max()
ax.plot([lo, hi], [qual["mean"].iloc[0], qual["mean"].iloc[-1]],
color=INK_SOFT,
        lw=1.4, ls="--", zorder=2, label="Constant-step line")
ax.yaxis.set_major_formatter(MONEY)
ax.set_xlabel("Overall quality (1–10)"); ax.set_ylabel("Mean sale price
(USD)")
ax.set_title("(a) Quality steps are not equal"); ax.legend(loc="upper left")

# panel 2 — widening spread (heteroscedasticity)
ax = axes[1]
```

```
ax.scatter(df["GrLivArea"], df[TARGET], s=12, color=BLUE, alpha=0.35,
linewidths=0)
ax.set_xlabel("Above-grade living area (ft²)"); ax.set_ylabel("Sale price
(USD)")
ax.yaxis.set_major_formatter(MONEY)
ax.set_title("(b) Fan shape: variance grows with size")

# panel 3 — log target straightens it out
ax = axes[2]
ax.scatter(df["GrLivArea"], np.log(df[TARGET]), s=12, color=AQUA,
alpha=0.35, linewidths=0)
ax.set_xlabel("Above-grade living area (ft²)"); ax.set_ylabel("log(Sale
price)")
ax.set_title("(c) log(price) is more even")

fig.suptitle("Signs of non-linearity in the raw target", x=0.005, ha="left",
             fontsize=12.5, fontweight="bold", color=INK)
fig.tight_layout(rect=[0, 0, 1, 0.94])
save(fig, "04_nonlinearity"); plt.show()
```




## Discussion questions — Step 2

**Q1. Which variable has the strongest correlation with `SalePrice`? Is it positive or negative?**

`OverallQual`, with **Pearson r = 0.791 — positive** ($p \approx 2.2 \times 10^{-313}$, i.e. indistinguishable from zero probability of arising by chance). On its own it explains $r^2 = 62.6\%$ of the variance in price. The full ranking is:

| Rank | Feature | Pearson r | Direction | Variance explained alone |
|---|---|---|---|---|
| 1 | `OverallQual` | **0.791** | positive | 62.6% |
| 2 | `GrLivArea` | 0.709 | positive | 50.2% |
| 3 | `GarageArea` | 0.623 | positive | 38.9% |
| 4 | `YearBuilt` | 0.523 | positive | 27.3% |

**All four are positive**, which matches intuition: better finish, more living space, a bigger garage and a newer build all raise the price. None of the four is negatively correlated with the target.

**Q2. Do you see any signs of non-linear relationships?**

Yes — three distinct signs:

1. **Spearman ρ exceeds Pearson r for every feature**, and most sharply for `YearBuilt` (ρ = 0.653 vs r = 0.523, a gap of **+0.130**). Spearman only asks whether the relationship is monotonic; Pearson asks whether it is a *straight line*. The gap is the signature of a monotonic but **curved** relationship — mean price is roughly flat (even dipping) for houses built before 1950 and then climbs steeply, from $148k for 1950–1970 builds to $245k for post-2000 builds.
2. **The quality ladder is convex** (panel a). Going from quality 5 to 6 adds about $28k to the mean price, whereas 8 to 9 adds about $93k. A single linear coefficient on `OverallQual` must average these very unequal steps, so it over-prices the bottom of the ladder and under-prices the top.
3. **The variance is not constant** (panel b). The price scatter around the trend widens steadily with size: within 0–1000 ft² the standard deviation of price is ≈ $26k, but above 2200 ft² it is ≈ $121k — a 4.6× increase. That "fan" shape is heteroscedasticity — it violates the constant-variance assumption of OLS, and it is why the model's dollar errors are largest for expensive houses (shown in Step 4). Taking logs of the target (panel c) largely removes it: skewness falls from **1.88 to 0.12**, and the log model is tested in Step 5.

Two further caveats visible in the plots: the four houses above 4,000 ft² are leverage points — two of them ( `Id` 524 and 1299, `SaleCondition = Partial`, unfinished sales of quality-10 homes) sold for far below what their size implies and distort any straight line; and `GarageArea = 0` for 81 houses (no garage), which puts a spike of points on the y-axis that a linear term fits poorly.

---

# Step 3 — Build the Linear Regression Model

**Tasks:** split the data 80/20, train a linear regression on the selected features, and display the intercept and coefficients.

In [13]:

```
X = df[FEATURES].copy()
y = df[TARGET].copy()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=RANDOM_STATE)

print(f"Training set: {X_train.shape[0]:,} rows
({X_train.shape[0]/len(X):.0%})")
print(f"Test set:     {X_test.shape[0]:,} rows
({X_test.shape[0]/len(X):.0%})")
print(f"Features:     {X_train.shape[1]}  ->  {', '.join(FEATURES)}")
print(f"\nMean SalePrice  train: {y_train.mean():>10,.0f}   test:
{y_test.mean():>10,.0f}"
      f"   (similar, so the split is not biased)")
```

```
Training set: 1,168 rows (80%)
Test set:     292 rows (20%)
Features:     4  ->  GrLivArea, OverallQual, YearBuilt, GarageArea

Mean SalePrice  train:    181,442   test:    178,840   (similar, so the split
is not biased)
```

In [14]:

```python
model = LinearRegression()
model.fit(X_train, y_train)

coefs = pd.DataFrame({
    "feature": FEATURES,
    "coefficient": model.coef_,
    "unit": ["per extra ft² of living area", "per +1 quality point",
             "per year newer", "per extra ft² of garage"],
})
coefs["coefficient"] = coefs["coefficient"].round(2)

print(f"Intercept (β₀): {model.intercept_:,.2f}\n")
print("Coefficients:")
print(coefs.to_string(index=False))
print(f"\nR² on the training set: {model.score(X_train, y_train):.4f}")
```

```
Intercept (β₀): -845,603.74

Coefficients:
    feature  coefficient                        unit
  GrLivArea        52.97 per extra ft² of living area
OverallQual     22708.55         per +1 quality point
  YearBuilt       395.57               per year newer
 GarageArea        57.70      per extra ft² of garage

R² on the training set: 0.7454
```

## The fitted regression equation

```
predicted SalePrice = -845,603.74
                    +      52.97  x  GrLivArea     (ft2 of above-
grade living area)
                    +  22,708.55  x  OverallQual   (quality points,
1-10)
                    +     395.57  x  YearBuilt     (construction
year)
                    +      57.70  x  GarageArea    (ft2 of garage)
```

In words: a house's predicted price starts from a (non-interpretable) baseline of –$845,604 and adds $52.97 for every square foot of above-grade living area, $22,708.55 for every one-point gain in overall quality, $395.57 for every year newer it was built, and $57.70 for every square foot of garage.

In [15]:

```python
# Worked example: predict one specific house by hand and with the model
house = X_test.iloc[[0]]
manual = model.intercept_ + float((house.values * model.coef_).sum())
print("Feature values for the first test house:")
print(house.T.to_string(header=False))
print(f"\nHand calculation : {manual:,.2f}")
print(f"model.predict()  : {model.predict(house)[0]:,.2f}")
print(f"Actual sale price: {y_test.iloc[0]:,.0f}")
```

```
Feature values for the first test house:
GrLivArea    1068
OverallQual     6
YearBuilt    1963
GarageArea    264

Hand calculation : 138,947.92
model.predict()  : 138,947.92
Actual sale price: 154,500
```

## Discussion questions — Step 3

**Q1. Write down the regression equation you obtained.**

```
predicted SalePrice = −845,603.74 + 52.97 x GrLivArea + 22,708.55 x
OverallQual
                                  + 395.57 x YearBuilt + 57.70 x
GarageArea
```

A note on the intercept: −$845,604 is the predicted price of a house with 0 ft² of living area, quality 0, built in year 0 and with no garage — a combination that cannot exist. It is a mathematical anchor that lets the plane pass through the data cloud (which sits at `YearBuilt ≈ 1971`, where the `YearBuilt` term alone contributes ≈ +$780k), **not** a meaningful price. This is why the standardised version in Step 5, whose intercept equals the mean price, is easier to read.

**Q2. How much does `SalePrice` increase/decrease for one unit change in `GrLivArea`?**

**It increases by $52.97 per additional square foot**, holding quality, year built and garage area constant. Equivalently:

- **+$5,297 per extra 100 ft²** of living space;
- ≈ **+$27,778 for one standard deviation** of living area (524 ft²).

Two qualifications matter. First, the *simple* regression of price on `GrLivArea` alone gives a slope of about $102/ft² — nearly twice as steep ($R^2$ = 0.484 on its own). The multiple-regression coefficient is smaller because big houses also tend to be higher quality (r = 0.59 between `GrLivArea` and `OverallQual`), so part of what raw size appeared to buy is really quality, and the model now credits it to `OverallQual`. Second, this is an **associational** statement under a linear, constant-effect assumption, not proof that adding 100 ft² to your own house would raise its sale price by exactly $5,297.

**Q3. Are there any features with negative coefficients? What do they imply?**

**No — all four coefficients are positive** (52.97, 22,708.55, 395.57, 57.70), so each feature pushes the prediction up, matching the positive correlations found in Step 2. The only negative number in the model is the intercept, which is not a feature effect.

Had a coefficient come out negative, it would mean one of three things:

1. **A genuine negative effect** — e.g. `OverallCond` can be negative in some specifications because a high "condition" score often marks an old house that has been patched up rather than a desirable new one.
2. **A suppression / collinearity artefact** — with correlated predictors, a feature's sign can flip once another absorbs the shared signal. If `TotalBsmtSF` and `1stFlrSF` (r ≈ 0.82) both enter a model, one frequently turns negative even though both raise price on their own. The honest reading is then *"conditional on the other, this adds nothing"*, not *"this lowers price"*.
3. **A sign of mis-specified form** — forcing a straight line on a curved relationship can produce a negative term that is really doing curvature repair.

This is worth watching in Step 5: the interaction model drives `GrLivArea`'s own coefficient from $46.07 down to $1.26/ft² — close to flipping negative — purely because the interaction term takes over the size effect.

In [16]:

```
# Supporting evidence for Q2 and Q3: the simple vs multiple slope on
GrLivArea
simple = LinearRegression().fit(X_train[["GrLivArea"]], y_train)
print(f"Simple regression   SalePrice ~ GrLivArea          : slope =
{simple.coef_[0]:7.2f} USD/ft²,"
      f"  R² = {simple.score(X_train[["GrLivArea"]], y_train):.4f}")
print(f"Multiple regression SalePrice ~ 4 features         : slope =
{model.coef_[0]:7.2f} USD/ft²,"
      f"  R² = {model.score(X_train, y_train):.4f}")
print(f"\nCorrelation between GrLivArea and OverallQual       : "
      f"{df['GrLivArea'].corr(df['OverallQual']):.3f}  <- why the slope
shrinks")
print(f"\nPer 100 ft²: {model.coef_[0]*100:,.0f} USD")
print(f"Per 1 SD of GrLivArea ({X_train['GrLivArea'].std():.0f} ft²): "
      f"{model.coef_[0]*X_train['GrLivArea'].std():,.0f} USD")
print(f"Any negative coefficients? {'yes' if (model.coef_ < 0).any() else
'no'}")
```

```
Simple regression   SalePrice ~ GrLivArea          : slope =  102.49 USD/ft²,
R² = 0.4839
Multiple regression SalePrice ~ 4 features         : slope =   52.97 USD/ft²,
R² = 0.7454

Correlation between GrLivArea and OverallQual       : 0.593  <- why the slope
shrinks

Per 100 ft²: 5,297 USD
Per 1 SD of GrLivArea (524 ft²): 27,778 USD
Any negative coefficients? no
```

# Step 4 — Evaluate the Model

**Tasks:** predict on the test set, report MSE and $R^2$, and plot actual against predicted prices.

In [17]:

```
y_pred = model.predict(X_test)

mse  = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae  = mean_absolute_error(y_test, y_pred)
r2   = r2_score(y_test, y_pred)
r2_train = model.score(X_train, y_train)
mape = (np.abs(y_pred - y_test) / y_test).mean() * 100

print("Test-set performance (292 houses)")
print("-" * 46)
print(f"MSE   (mean squared error) : {mse:>18,.0f}")
print(f"RMSE  (√MSE, in USD)       : {rmse:>18,.0f}")
print(f"MAE   (mean absolute error): {mae:>18,.0f}")
print(f"MAPE  (mean abs % error)   : {mape:>17.1f} %")
print(f"R²    (test)               : {r2:>18.4f}")
print(f"R²    (train)              : {r2_train:>18.4f}")
print(f"\nFor scale: mean test price = {y_test.mean():,.0f} USD,"
      f" so RMSE is {rmse/y_test.mean()*100:.1f}% of the average house
price.")

cv = cross_val_score(LinearRegression(), X, y, cv=KFold(5, shuffle=True,
random_state=RANDOM_STATE), scoring="r2")
print(f"\n5-fold cross-validated R² on all 1,460 rows: "
      f"{cv.mean():.4f} ± {cv.std():.4f}   (folds: {np.round(cv, 4)})")
```

```
Test-set performance (292 houses)
----------------------------------------------
MSE   (mean squared error) :      1,767,706,248
RMSE  (√MSE, in USD)       :             42,044
MAE   (mean absolute error):             26,879
MAPE  (mean abs % error)   :              16.2 %
R²    (test)               :             0.7695
R²    (train)              :             0.7454

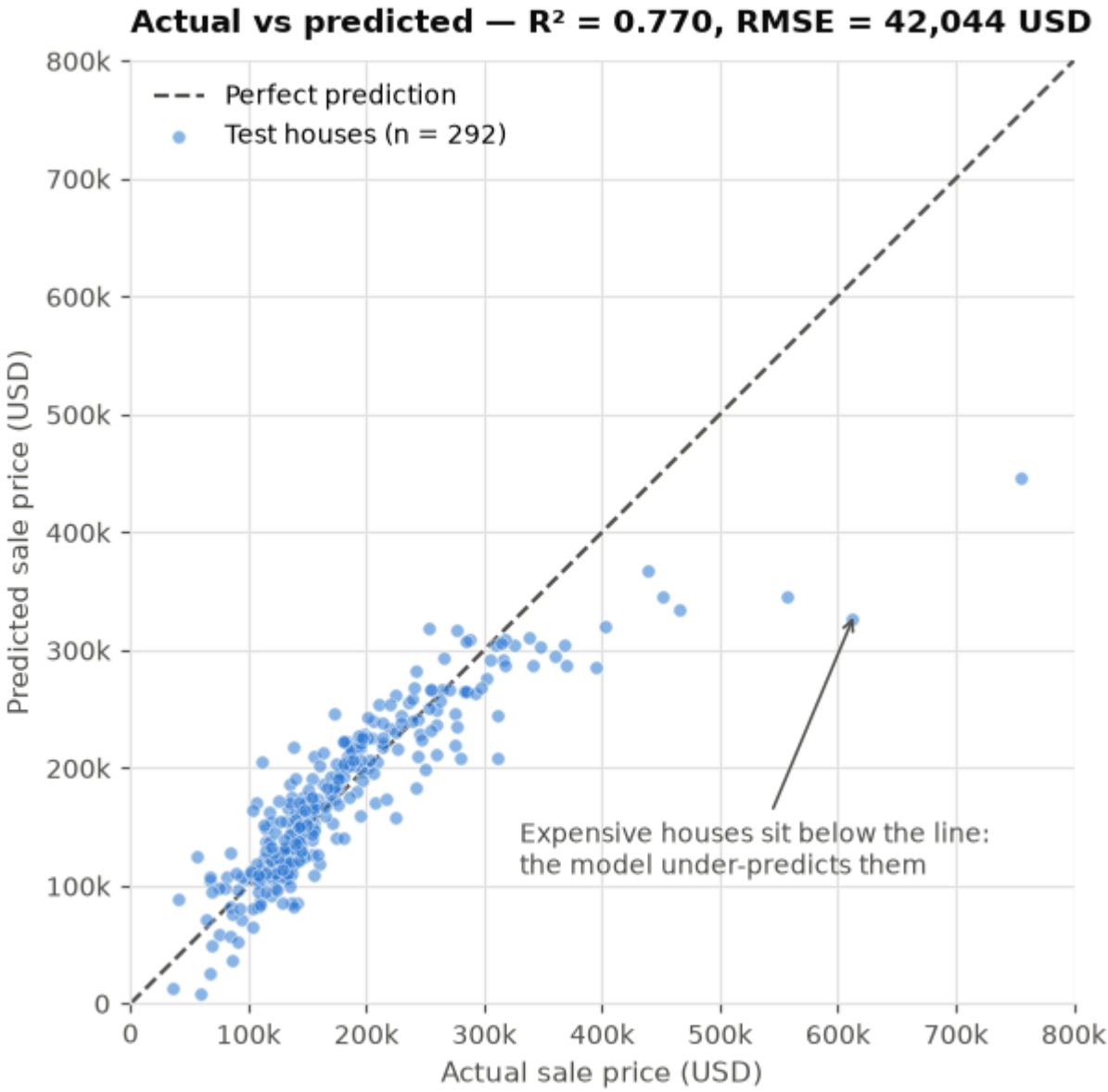
For scale: mean test price = 178,840 USD, so RMSE is 23.5% of the average
house price.

5-fold cross-validated R² on all 1,460 rows: 0.7362 ± 0.1027   (folds: [0.7695
0.7987 0.5319 0.783  0.7977])
```

In [18]:

```
# ---- Actual vs predicted --------------------------------------------
-
fig, ax = plt.subplots(figsize=(6.6, 6.0))
lims = [0, 800_000]
ax.plot(lims, lims, color=INK_SOFT, lw=1.4, ls="--", zorder=2,
```

```
label="Perfect prediction")
ax.scatter(y_test, y_pred, s=22, color=BLUE, alpha=0.55, linewidths=0.5,
           edgecolors="white", zorder=3, label="Test houses (n = 292)")
ax.set_xlim(lims); ax.set_ylim(lims); ax.set_aspect("equal")
ax.xaxis.set_major_formatter(MONEY); ax.yaxis.set_major_formatter(MONEY)
ax.set_xlabel("Actual sale price (USD)"); ax.set_ylabel("Predicted sale
price (USD)")
ax.set_title(f"Actual vs predicted — R² = {r2:.3f}, RMSE = {rmse:,.0f} USD")
ax.legend(loc="upper left")
ax.annotate("Expensive houses sit below the line:\nthe model under-predicts
them",
            xy=(615_000, 332_000), xytext=(330_000, 110_000), fontsize=9,
color=INK_SOFT,
            arrowprops=dict(arrowstyle="->", color=INK_SOFT, lw=1))
save(fig, "05_actual_vs_predicted"); plt.show()
```




In [19]:

```
# ---- Residual diagnostics ----------------------------------------------
-
resid = y_test - y_pred
```

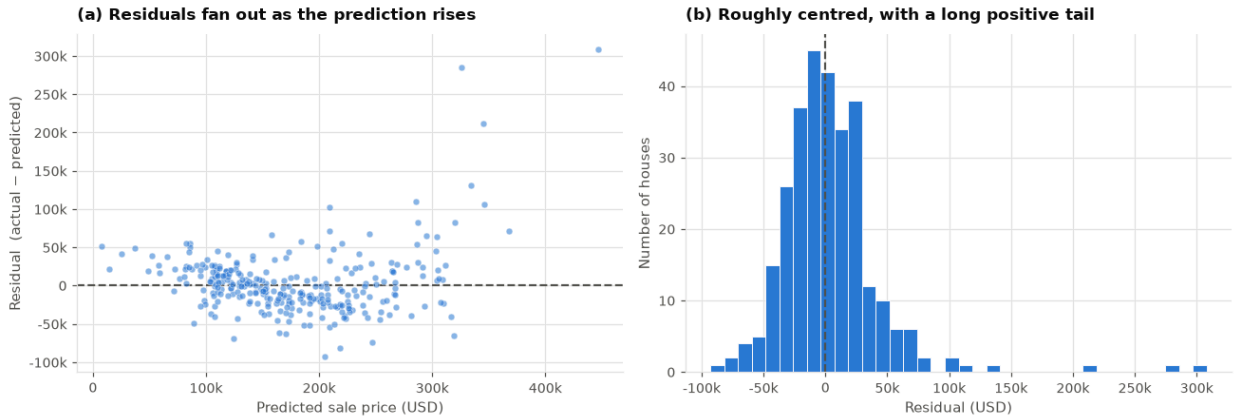
```
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.2))

ax = axes[0]
ax.axhline(0, color=INK_SOFT, lw=1.4, ls="--", zorder=2)
ax.scatter(y_pred, resid, s=20, color=BLUE, alpha=0.55, linewidths=0.5,
           edgecolors="white", zorder=3)
ax.xaxis.set_major_formatter(MONEY); ax.yaxis.set_major_formatter(MONEY)
ax.set_xlabel("Predicted sale price (USD)"); ax.set_ylabel("Residual
(actual - predicted)")
ax.set_title("(a) Residuals fan out as the prediction rises")

ax = axes[1]
ax.hist(resid, bins=36, color=BLUE, edgecolor="white", linewidth=0.6)
ax.axvline(0, color=INK_SOFT, lw=1.4, ls="--")
ax.xaxis.set_major_formatter(MONEY)
ax.set_xlabel("Residual (USD)"); ax.set_ylabel("Number of houses")
ax.set_title("(b) Roughly centred, with a long positive tail")

fig.tight_layout()
save(fig, "06_residuals"); plt.show()

print(f"Residual mean   : {resid.mean():>12,.0f} USD  (small vs RMSE: little
overall bias)")
print(f"Residual std    : {resid.std():>12,.0f} USD")
print(f"Skewness        : {resid.skew():>12.3f}        (long right tail)")
print(f"Largest under-prediction: {resid.max():>12,.0f} USD")
print(f"Largest over-prediction : {resid.min():>12,.0f} USD")
```




```
Residual mean   :        3,440 USD  (small vs RMSE: little overall bias)
Residual std    :       41,975 USD
Skewness        :        2.940        (long right tail)
Largest under-prediction:      308,144 USD
Largest over-prediction :      -92,924 USD
```

In [20]:

```
# ---- Where does the model do badly? Error by price quartile ------------
-
ev = pd.DataFrame({"actual": y_test.values, "predicted": y_pred})
ev["error"] = ev["predicted"] - ev["actual"]          # positive = over-
predicted
ev["abs_pct_error"] = ev["error"].abs() / ev["actual"] * 100
ev["bucket"] = pd.qcut(ev["actual"], 4,
                       labels=["Q1 lowest\n(≤127k)", "Q2\n(127–154k)",
                               "Q3\n(154–209k)", "Q4 highest\n(>209k)"])
```

```
by_q = ev.groupby("bucket", observed=True).agg(
    n=("actual", "size"),
    mean_actual=("actual", "mean"),
    mean_predicted=("predicted", "mean"),
    mean_error=("error", "mean"),
    rmse=("error", lambda s: np.sqrt((s ** 2).mean())),
    mape=("abs_pct_error", "mean"),
).round(1)
print("Quartile edges:", [f"{v:,.0f}" for v in ev['actual'].quantile([0,
.25, .5, .75, 1]).values])
by_q
```

```
Quartile edges: ['35,311', '127,000', '154,150', '209,175', '755,000']
```

Out[20]:

| bucket | n | mean_actual | mean_predicted | mean_error | rmse | mape |
|---|---|---|---|---|---|---|
| **Q1 lowest\n(≤127k)** | 75 | 101607.4 | 105274.6 | 3667.1 | 28466.5 | 25.1 |
| **Q2\n(127-154k)** | 71 | 139672.6 | 139976.2 | 303.6 | 24962.4 | 14.1 |
| **Q3\n(154-209k)** | 73 | 179376.4 | 191218.0 | 11841.6 | 26307.3 | 12.3 |
| **Q4 highest\n(>209k)** | 73 | 295745.7 | 266080.4 | -29665.3 | 70286.2 | 12.8 |

In [21]:

```
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.2))
labels = [str(b).replace("\n", " ") for b in by_q.index]
xs = np.arange(len(by_q))

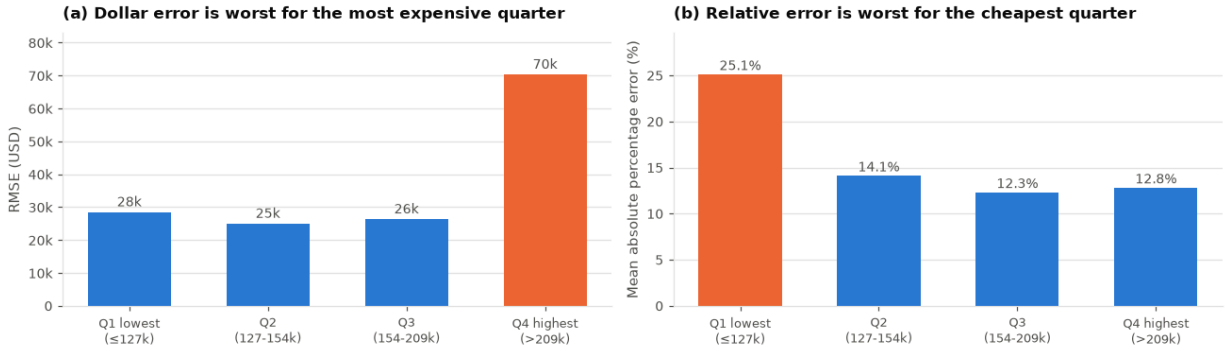
ax = axes[0]
cols = [ORANGE if v == by_q["rmse"].max() else BLUE for v in by_q["rmse"]]
ax.bar(xs, by_q["rmse"], color=cols, width=0.6)
for x, v in zip(xs, by_q["rmse"]):
    ax.text(x, v + 1800, f"{v/1000:,.0f}k", ha="center", fontsize=9,
color=INK_SOFT)
ax.set_xticks(xs, by_q.index, fontsize=8.5)
ax.yaxis.set_major_formatter(MONEY); ax.set_ylabel("RMSE (USD)")
ax.set_ylim(0, by_q["rmse"].max() * 1.18)
ax.set_title("(a) Dollar error is worst for the most expensive quarter")
ax.grid(axis="x", visible=False)

ax = axes[1]
cols = [ORANGE if v == by_q["mape"].max() else BLUE for v in by_q["mape"]]
ax.bar(xs, by_q["mape"], color=cols, width=0.6)
for x, v in zip(xs, by_q["mape"]):
    ax.text(x, v + 0.5, f"{v:.1f}%", ha="center", fontsize=9,
color=INK_SOFT)
ax.set_xticks(xs, by_q.index, fontsize=8.5)
ax.set_ylabel("Mean absolute percentage error (%)")
ax.set_ylim(0, by_q["mape"].max() * 1.18)
ax.set_title("(b) Relative error is worst for the cheapest quarter")
ax.grid(axis="x", visible=False)

fig.suptitle("Two different failure modes at the two ends of the price
range",
```

```
            x=0.005, ha="left", fontsize=12.5, fontweight="bold",
color=INK)
fig.tight_layout(rect=[0, 0, 1, 0.93])
save(fig, "07_error_by_price_band"); plt.show()
```

**Two different failure modes at the two ends of the price range**




In [22]:

```
# The five worst misses
worst =
ev.reindex(ev["error"].abs().sort_values(ascending=False).index).head(5)
print("Five largest absolute errors on the test set:")
print(worst[["actual", "predicted", "error",
"abs_pct_error"]].round(0).to_string(index=False))
```

```
Five largest absolute errors on the test set:
 actual  predicted     error  abs_pct_error
 755000   446856.0 -308144.0           41.0
 611657   325997.0 -285660.0           47.0
 556581   345109.0 -211472.0           38.0
 465000   333996.0 -131004.0           28.0
 395000   285720.0 -109280.0           28.0
```

## Discussion questions — Step 4

**Q1. What is the $R^2$ value? What does it indicate about your model's performance?**

**$R^2$ = 0.7695 on the held-out test set** (0.7454 on the training set; 5-fold cross-validated $R^2$ = 0.7362 ± 0.1027). The accompanying errors are **MSE = 1,767,706,248**, i.e. **RMSE = $42,044** and **MAE = $26,879** (MAPE = 16.2%, and RMSE is 23.5% of the mean test price).

What it means:

- **76.95% of the variance in sale price is explained** by just four features; the remaining 23% is down to everything the model cannot see — neighbourhood, lot shape, basement finish, kitchen quality, market timing — plus genuine noise in how individual sales are negotiated.
- **Test $R^2$ ≥ train $R^2$** (0.7695 vs 0.7454), so the model is **not overfitting**. The cross-validated folds do vary (0.53 to 0.80): the weak fold is the one that happens to contain

several of the extreme luxury houses, which is itself evidence of how much the tail of the price distribution drives the score — any single train/test split of this dataset should be read with that sensitivity in mind. With four parameters and 1,168 training rows that is expected; if anything the model is slightly *under*-fit (high bias), which is the normal price of a simple linear specification.

- **In practical terms the accuracy is moderate, not production-grade.** A typical prediction is off by about $27k (MAE) on a mean price of $182k — roughly 15%. That is useful for a rough automated valuation or a sanity check on an asking price, but far too loose to settle a transaction. The headroom is large: this model uses 4 of the 79 available predictors and assumes a straight line, whereas the full-feature gradient-boosted models that top the Kaggle leaderboard for this dataset cut the error substantially further.
- **$R^2$ alone would hide the structure of the errors.** The residual plot shows the errors are heteroscedastic and strongly right-skewed (skew = 2.94), so the single $R^2$ number overstates how trustworthy the model is for any individual expensive house.

**Q2. In which price range (low/high) does the model perform poorly?**

Both ends fail, but in **opposite ways** — which is why the two panels above use different measures:

| Quartile | Price range | n | Mean error (signed) | RMSE | MAPE |
|---|---|---|---|---|---|
| Q1 lowest | ≤ $127,000 | 75 | **+$3,667** (over-predicts) | $28,467 | **25.1%** |
| Q2 | $127k–154k | 71 | +$304 | $24,962 | 14.1% |
| Q3 | $154k–209k | 73 | +$11,842 | $26,307 | 12.3% |
| Q4 highest | > $209,175 | 73 | **−$29,665** (under-predicts) | **$70,286** | 12.8% |

- **In dollar terms the model is worst on expensive houses.** The top quartile's RMSE ($70,286) is **2.6× the average of the other three quartiles** ($25–28k), and the error is systematically *negative*: the model under-prices the top quartile by $29,665 on average. All five of the largest misses are luxury homes, led by a $755,000 house predicted at $446,856 (−$308,144, a 41% shortfall). The cause is structural: a straight line cannot reproduce the convex quality ladder of Step 2, and the premium features of a luxury home (pool, 3-car garage, premium neighbourhood, superb kitchen) are simply absent from the four features.
- **In percentage terms the model is worst on cheap houses.** The bottom quartile carries a MAPE of **25.1%** — double the middle of the market — and a slight upward bias. The floor of the straight line is too high for small, old, low-quality houses: with four positive coefficients the model has no way to predict a price below a certain level, so the cheapest homes get pulled up toward the mean.
- **The middle of the market ($127k–$209k) is where the model is reliable**: near-zero bias in Q2 and 12–14% relative error. That is also where the bulk of the data sits, so OLS — which minimises *squared* error — optimises for exactly this region and tolerates the tails.

---

# Step 5 — Extension

All three suggested extensions are carried out and compared against the Step 3 baseline on the **same** 80/20 split, so every $R^2$ below is measured on the same 292 held-out houses:

1. **More features** — add `TotalBsmtSF` and `LotArea` ( `GarageArea` is already in).
2. **Standardisation** — z-score the predictors and re-read the coefficients.
3. **Interaction term** — `GrLivArea × OverallQual`.

Plus a fourth, motivated by the non-linearity found in Step 2: **modelling `log(SalePrice)`** instead of the raw price. Multicollinearity is then measured with variance inflation factors.

## 5.1 — Adding more features

In [23]:

```
EXTENDED = ["GrLivArea", "OverallQual", "YearBuilt", "GarageArea",
"TotalBsmtSF", "LotArea"]

Xe = df[EXTENDED].copy()
Xe_train, Xe_test, ye_train, ye_test = train_test_split(
    Xe, y, test_size=0.20, random_state=RANDOM_STATE)
assert (ye_test.index == y_test.index).all()          # identical split as
Step 3

model_ext = LinearRegression().fit(Xe_train, y_train)
pred_ext  = model_ext.predict(Xe_test)

print(f"Intercept: {model_ext.intercept_:,.2f}\n")
print(pd.DataFrame({"feature": EXTENDED,
                    "coefficient":
model_ext.coef_.round(3)}).to_string(index=False))
print(f"\nTest R²  : {r2_score(y_test, pred_ext):.4f}   (baseline 4
features: {r2:.4f})")
print(f"Test RMSE: {np.sqrt(mean_squared_error(y_test, pred_ext)):,.0f} USD"
      f"   (baseline: {rmse:,.0f} USD)")
```

```
Intercept: -790,554.14

    feature  coefficient
  GrLivArea       46.069
OverallQual    21856.486
  YearBuilt      364.654
 GarageArea       45.171
TotalBsmtSF       19.470
    LotArea        0.648

Test R²  : 0.7957   (baseline 4 features: 0.7695)
Test RMSE: 39,584 USD   (baseline: 42,044 USD)
```

## 5.2 — Standardising the predictors

In [24]:

```
scaler = StandardScaler().fit(Xe_train)
model_std = LinearRegression().fit(scaler.transform(Xe_train), y_train)
pred_std  = model_std.predict(scaler.transform(Xe_test))

compare = pd.DataFrame({
    "feature": EXTENDED,
    "raw_coefficient": model_ext.coef_.round(3),
    "unit_of_raw_coef": ["USD per ft²", "USD per quality point", "USD per
year",
                         "USD per ft²", "USD per ft²", "USD per ft²"],
    "standardised_coefficient": model_std.coef_.round(0),
})
compare["rank_by_raw"]  =
compare["raw_coefficient"].abs().rank(ascending=False).astype(int)
compare["rank_by_std"]  =
compare["standardised_coefficient"].abs().rank(ascending=False).astype(int)
print(compare.to_string(index=False))

print(f"\nIntercept of the standardised model: {model_std.intercept_:,.2f}")
print(f"Mean SalePrice in the training set  : {y_train.mean():,.2f}   <-
identical, by construction")
print(f"\nTest R² raw        : {r2_score(y_test, pred_ext):.6f}")
print(f"Test R² standardised: {r2_score(y_test, pred_std):.6f}   <-
unchanged: scaling cannot alter the fit")
```

```
    feature  raw_coefficient      unit_of_raw_coef  standardised_coefficient
rank_by_raw  rank_by_std
  GrLivArea           46.069           USD per ft²                   24150.0
3            2
OverallQual        21856.486 USD per quality point                   29879.0
1            1
  YearBuilt          364.654          USD per year                   11181.0
2            3
 GarageArea           45.171           USD per ft²                    9531.0
4            4
TotalBsmtSF           19.470           USD per ft²                    8576.0
5            5
    LotArea            0.648           USD per ft²                    6970.0
6            6

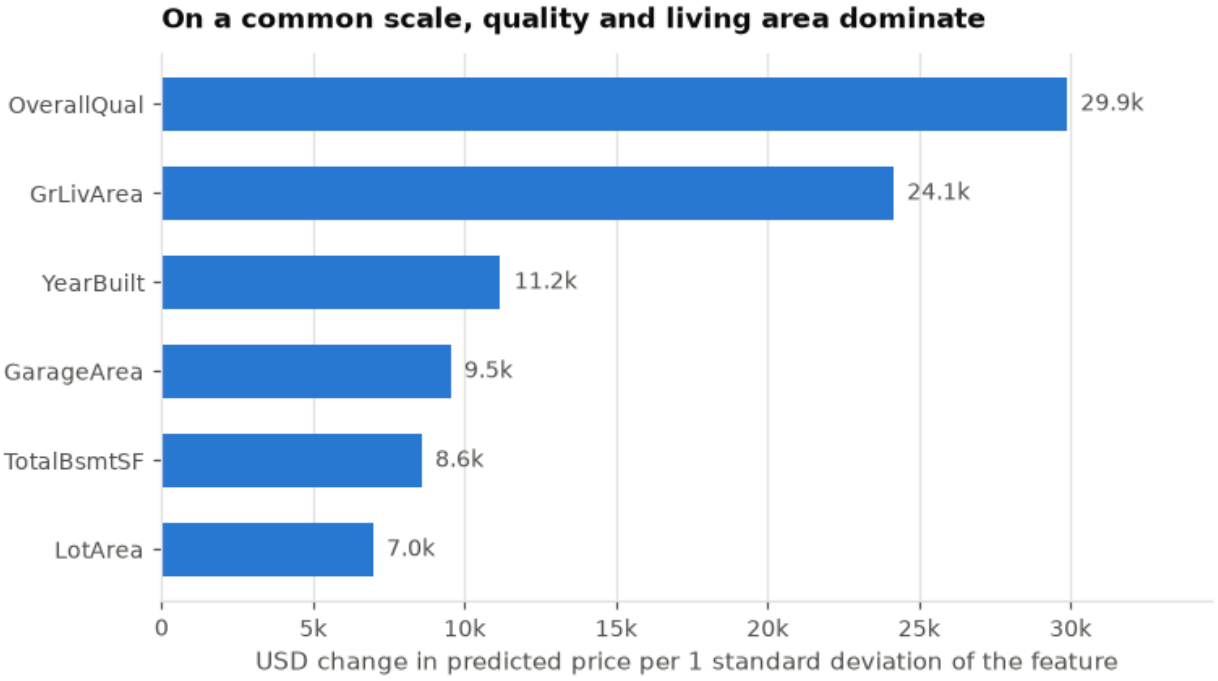
Intercept of the standardised model: 181,441.54
Mean SalePrice in the training set  : 181,441.54   <- identical, by
construction

Test R² raw        : 0.795724
Test R² standardised: 0.795724   <- unchanged: scaling cannot alter the fit
```

In [25]:

```
fig, ax = plt.subplots(figsize=(7.6, 4.0))
o = compare.sort_values("standardised_coefficient")
ax.barh(o["feature"], o["standardised_coefficient"], color=BLUE, height=0.6)
```

```
for f, v in zip(o["feature"], o["standardised_coefficient"]):
    ax.text(v + 450, f, f"{v/1000:,.1f}k", va="center", fontsize=9,
color=INK_SOFT)
ax.set_xlim(0, o["standardised_coefficient"].max() * 1.16)
ax.xaxis.set_major_formatter(MONEY)
ax.set_xlabel("USD change in predicted price per 1 standard deviation of the
feature")
ax.set_title("On a common scale, quality and living area dominate")
ax.grid(axis="y", visible=False)
save(fig, "08_standardised_coefficients"); plt.show()
```




## 5.3 — Interaction term: `GrLivArea × OverallQual`

In [26]:

```
Xi_train = Xe_train.copy(); Xi_test = Xe_test.copy()
for frame in (Xi_train, Xi_test):
    frame["GrLivArea_x_OverallQual"] = frame["GrLivArea"] *
frame["OverallQual"]

model_int = LinearRegression().fit(Xi_train, y_train)
pred_int  = model_int.predict(Xi_test)

print(f"Intercept: {model_int.intercept_:,.2f}\n")
print(pd.DataFrame({"feature": Xi_train.columns,
                    "coefficient":
model_int.coef_.round(3)}).to_string(index=False))
print(f"\nTest R²  : {r2_score(y_test, pred_int):.4f}")
print(f"Test RMSE: {np.sqrt(mean_squared_error(y_test, pred_int)):,.0f}
USD")

# What the interaction means: the marginal value of a square foot by quality
level
```

```
b_area, b_inter = model_int.coef_[0], model_int.coef_[-1]
print("\nMarginal value of one extra ft² of living area, by quality level:")
for q_level in (4, 6, 8, 10):
    print(f"  OverallQual = {q_level:>2} :  {b_area + b_inter *
q_level:6.2f} USD per ft²")
```

```
Intercept: -746,100.45

                feature  coefficient
              GrLivArea        1.264
            OverallQual    12058.737
              YearBuilt      376.177
             GarageArea       44.960
            TotalBsmtSF       15.988
                LotArea        0.646
GrLivArea_x_OverallQual        6.662

Test R²  : 0.8239
Test RMSE: 36,752 USD

Marginal value of one extra ft² of living area, by quality level:
  OverallQual =  4 :   27.91 USD per ft²
  OverallQual =  6 :   41.24 USD per ft²
  OverallQual =  8 :   54.56 USD per ft²
  OverallQual = 10 :   67.88 USD per ft²
```

## 5.4 — Modelling `log(SalePrice)` (addressing the non-linearity from Step 2)

In [27]:

```
model_log = LinearRegression().fit(Xe_train, np.log(y_train))
pred_log  = np.exp(model_log.predict(Xe_test))        # back to USD for a
fair comparison

print(f"Test R² (back on the USD scale): {r2_score(y_test, pred_log):.4f}")
print(f"Test RMSE: {np.sqrt(mean_squared_error(y_test, pred_log)):,.0f} USD"
      f"   |   MAE: {mean_absolute_error(y_test, pred_log):,.0f} USD")

res_log = pd.DataFrame({"actual": y_test.values, "predicted": pred_log})
res_log["abs_pct"] = (res_log.predicted - res_log.actual).abs() /
res_log.actual * 100
bucket = pd.qcut(res_log["actual"], 4, labels=["Q1", "Q2", "Q3", "Q4"])
print("\nMAPE by price quartile, log model vs baseline:")
print(pd.DataFrame({
    "log_model_mape": res_log.groupby(bucket, observed=True)
["abs_pct"].mean().round(1),
    "baseline_mape":  by_q["mape"].values,
}).to_string())
```

```
Test R² (back on the USD scale): 0.8642
Test RMSE: 32,269 USD   |   MAE: 20,302 USD

MAPE by price quartile, log model vs baseline:
        log_model_mape  baseline_mape
actual
Q1                19.6           25.1
Q2                 8.8           14.1
Q3                 8.8           12.3
Q4                11.7           12.8
```

## 5.5 — Multicollinearity

In [28]:

```
def vif_table(frame):
    # Variance inflation factor: VIF_j = 1 / (1 - R2_j), where R2_j comes
from
    # regressing feature j on all the other features.
    out = {}
    for col in frame.columns:
        others = frame.drop(columns=[col])
        r2_j = LinearRegression().fit(others, frame[col]).score(others,
frame[col])
        out[col] = 1 / (1 - r2_j)
    return pd.Series(out, name="VIF").round(2)

Xi_all = df[EXTENDED].copy()
Xi_all["GrLivArea_x_OverallQual"] = Xi_all["GrLivArea"] *
Xi_all["OverallQual"]

# Centring the two parent features before multiplying them
Xc_all = df[EXTENDED].copy()
Xc_all["GrLivArea_x_OverallQual_centred"] = (
    (df["GrLivArea"] - df["GrLivArea"].mean()) * (df["OverallQual"] -
df["OverallQual"].mean()))

vifs = pd.concat([
    vif_table(df[FEATURES]).rename("baseline (4 feats)"),
    vif_table(df[EXTENDED]).rename("extended (6 feats)"),
    vif_table(Xi_all).rename("with raw interaction"),
    vif_table(Xc_all).rename("with centred interaction"),
], axis=1)
print("Variance inflation factors  (rule of thumb: >5 is a concern, >10 is
severe)\n")
vifs
```

```
Variance inflation factors  (rule of thumb: >5 is a concern, >10 is severe)
```

Out[28]:

| | baseline (4 feats) | extended (6 feats) | with raw interaction | with centred interaction |
|---|---|---|---|---|
| **GrLivArea** | 1.74 | 1.86 | 15.98 | 1.97 |
| **OverallQual** | 2.40 | 2.52 | 7.16 | 2.53 |
| **YearBuilt** | 1.70 | 1.72 | 1.73 | 1.73 |
| **GarageArea** | 1.66 | 1.74 | 1.74 | 1.74 |
| **TotalBsmtSF** | NaN | 1.63 | 1.67 | 1.67 |
| **LotArea** | NaN | 1.13 | 1.13 | 1.13 |
| **GrLivArea_x_OverallQual** | NaN | NaN | 30.59 | NaN |
| **GrLivArea_x_OverallQual_centred** | NaN | NaN | NaN | 1.19 |

In [29]:

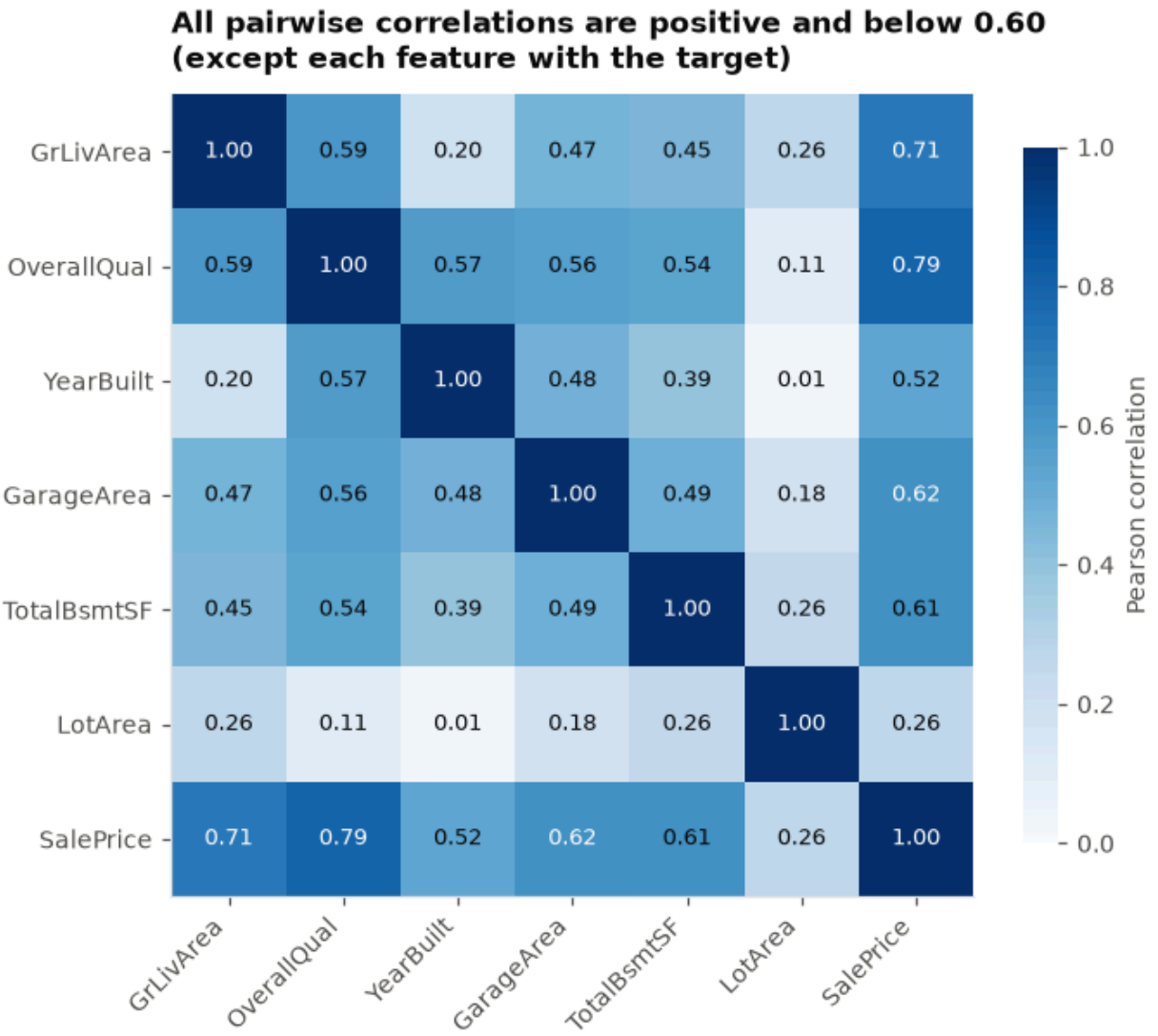
```
# Does centring the interaction change the predictions? (No — only the
VIFs.)
Xc_train, Xc_test, _, _ = train_test_split(Xc_all, y, test_size=0.20,
random_state=RANDOM_STATE)
model_cen = LinearRegression().fit(Xc_train, y_train)
print(f"Raw interaction     — test R²: {r2_score(y_test, pred_int):.4f}")
print(f"Centred interaction — test R²: {r2_score(y_test,
model_cen.predict(Xc_test)):.4f}"
      "   <— same fit, far lower VIFs")
```

```
Raw interaction     — test R²: 0.8239
Centred interaction — test R²: 0.8239   <— same fit, far lower VIFs
```

In [30]:

```
# ---- Correlation heat map among the predictors ----------------------------
cm = df[EXTENDED + [TARGET]].corr()

fig, ax = plt.subplots(figsize=(6.8, 5.8))
im = ax.imshow(cm.values, cmap="Blues", vmin=0, vmax=1)
ax.set_xticks(range(len(cm)), cm.columns, rotation=45, ha="right",
fontsize=9)
ax.set_yticks(range(len(cm)), cm.columns, fontsize=9)
for i in range(len(cm)):
    for j in range(len(cm)):
        v = cm.values[i, j]
        ax.text(j, i, f"{v:.2f}", ha="center", va="center", fontsize=8.5,
                color="white" if v > 0.62 else INK)
ax.set_title("All pairwise correlations are positive and below 0.60\n(except
each feature with the target)")
ax.grid(visible=False)
cb = fig.colorbar(im, ax=ax, shrink=0.82); cb.outline.set_visible(False)
cb.set_label("Pearson correlation", fontsize=9, color=INK_SOFT)
save(fig, "09_correlation_heatmap"); plt.show()
```




## 5.6 — Model comparison

In [31]:

```
def scores(name, predictions, n_features, fitted, X_tr):
    return {"model": name, "n_features": n_features,
            "train_R2": round(fitted.score(X_tr, y_train), 4),
            "test_R2": round(r2_score(y_test, predictions), 4),
            "test_MSE": round(mean_squared_error(y_test, predictions)),
            "test_RMSE": round(np.sqrt(mean_squared_error(y_test,
predictions))),
            "test_MAE": round(mean_absolute_error(y_test, predictions))}

summary = pd.DataFrame([
    scores("1. Baseline (Step 3)", y_pred,    4, model,     X_train),
    scores("2. + TotalBsmtSF, LotArea", pred_ext, 6, model_ext, Xe_train),
    scores("3. Standardised (6 feats)", pred_std, 6, model_std,
scaler.transform(Xe_train)),
    scores("4. + GrLivArea × OverallQual", pred_int, 7, model_int,
Xi_train),
])
log_row = {"model": "5. log(SalePrice), 6 feats", "n_features": 6,
           "train_R2": round(model_log.score(Xe_train, np.log(y_train)), 4),
           "test_R2": round(r2_score(y_test, pred_log), 4),
```

```
            "test_MSE": round(mean_squared_error(y_test, pred_log)),
            "test_RMSE": round(np.sqrt(mean_squared_error(y_test,
pred_log))),
            "test_MAE": round(mean_absolute_error(y_test, pred_log))}
summary = pd.concat([summary, pd.DataFrame([log_row])], ignore_index=True)
summary["ΔR2_vs_baseline"] = (summary["test_R2"] - summary.loc[0,
"test_R2"]).round(4)
summary
```

Out[31]:

| | model | n_features | train_R2 | test_R2 | test_MSE | test_RMSE | test_MAE | ΔI |
|---|---|---|---|---|---|---|---|---|
| **0** | 1. Baseline (Step 3) | 4 | 0.7454 | 0.7695 | 1767706248 | 42044 | 26879 | |
| **1** | 2. + TotalBsmtSF, LotArea | 6 | 0.7636 | 0.7957 | 1566865739 | 39584 | 24332 | |
| **2** | 3. Standardised (6 feats) | 6 | 0.7636 | 0.7957 | 1566865739 | 39584 | 24332 | |
| **3** | 4. + GrLivArea × OverallQual | 7 | 0.7701 | 0.8239 | 1350725783 | 36752 | 22594 | |
| **4** | 5. log(SalePrice), 6 feats | 6 | 0.8048 | 0.8642 | 1041308132 | 32269 | 20302 | |

In [32]:

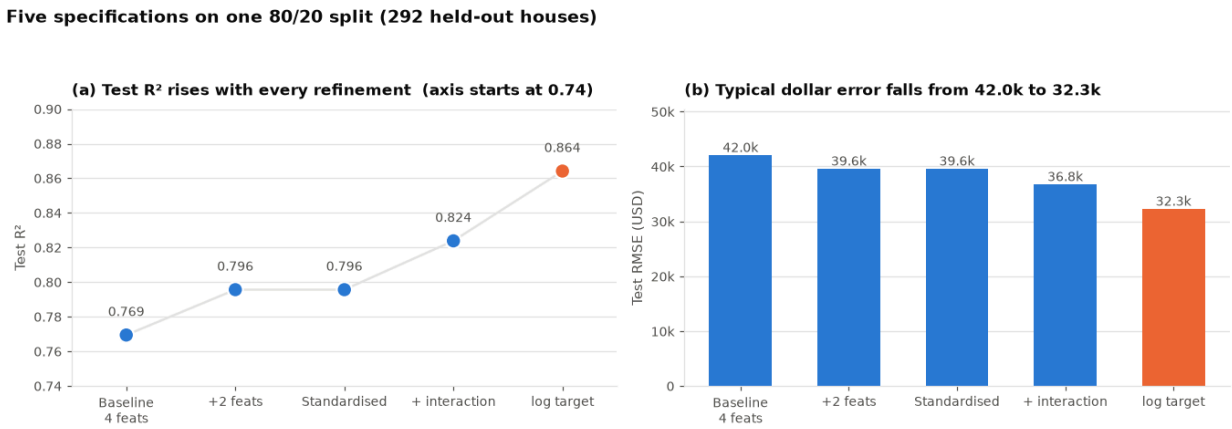
```
fig, axes = plt.subplots(1, 2, figsize=(12.4, 4.4))
short = ["Baseline\n4 feats", "+2 feats", "Standardised", "+ interaction",
"log target"]

ax = axes[0]
# A dot plot, not bars: the axis does not start at zero, so a bar would
# exaggerate the differences between the five models.
best = summary["test_R2"].max()
xs_r2 = np.arange(len(summary))
cols = [ORANGE if v == best else BLUE for v in summary["test_R2"]]
ax.plot(xs_r2, summary["test_R2"], color=GRID, lw=1.6, zorder=2)
ax.scatter(xs_r2, summary["test_R2"], s=120, color=cols, zorder=3,
           edgecolors="white", linewidths=1.5)
for i, v in enumerate(summary["test_R2"]):
    ax.text(i, v + 0.011, f"{v:.3f}", ha="center", fontsize=9,
color=INK_SOFT)
ax.set_xticks(xs_r2, short, fontsize=9)
ax.set_xlim(-0.5, len(summary) - 0.5)
ax.set_ylim(0.74, 0.90); ax.set_ylabel("Test R²")
ax.set_title("(a) Test R² rises with every refinement  (axis starts at
0.74)")
ax.grid(axis="x", visible=False)

ax = axes[1]
worst = summary["test_RMSE"].max()
cols = [ORANGE if v == summary["test_RMSE"].min() else BLUE for v in
summary["test_RMSE"]]
ax.bar(np.arange(len(summary)), summary["test_RMSE"], color=cols,
```

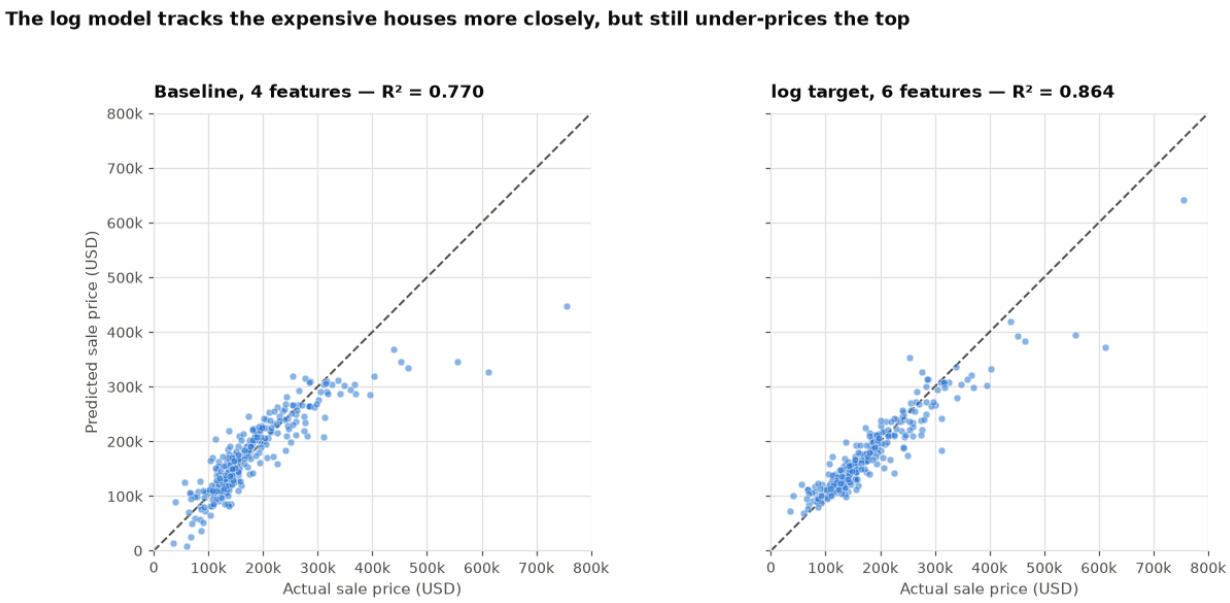
```
width=0.58)
for i, v in enumerate(summary["test_RMSE"]):
    ax.text(i, v + 600, f"{v/1000:,.1f}k", ha="center", fontsize=9,
color=INK_SOFT)
ax.set_xticks(np.arange(len(summary)), short, fontsize=9)
ax.yaxis.set_major_formatter(MONEY); ax.set_ylabel("Test RMSE (USD)")
ax.set_ylim(0, worst * 1.2)
ax.set_title("(b) Typical dollar error falls from 42.0k to 32.3k")
ax.grid(axis="x", visible=False)

fig.suptitle("Five specifications on one 80/20 split (292 held-out houses)",
             x=0.005, ha="left", fontsize=12.5, fontweight="bold",
color=INK)
fig.tight_layout(rect=[0, 0, 1, 0.92])
save(fig, "10_model_comparison"); plt.show()
```




In [33]:

```
# ---- Does the best model fix the high-end failure found in Step 4? -------
fig, axes = plt.subplots(1, 2, figsize=(11.6, 5.4), sharex=True,
sharey=True)
lims = [0, 800_000]
for ax, (ttl, pr) in zip(axes, [(f"Baseline, 4 features — R² = {r2:.3f}",
y_pred),
                                (f"log target, 6 features — R² =
{r2_score(y_test, pred_log):.3f}", pred_log)]):
    ax.plot(lims, lims, color=INK_SOFT, lw=1.3, ls="--", zorder=2)
    ax.scatter(y_test, pr, s=20, color=BLUE, alpha=0.55, linewidths=0.5,
               edgecolors="white", zorder=3)
    ax.set_xlim(lims); ax.set_ylim(lims); ax.set_aspect("equal")
    ax.xaxis.set_major_formatter(MONEY); ax.yaxis.set_major_formatter(MONEY)
    ax.set_xlabel("Actual sale price (USD)"); ax.set_title(ttl)
axes[0].set_ylabel("Predicted sale price (USD)")
fig.suptitle("The log model tracks the expensive houses more closely, but
still under-prices the top",
             x=0.005, ha="left", fontsize=12, fontweight="bold", color=INK)
fig.tight_layout(rect=[0, 0, 1, 0.94])
save(fig, "11_baseline_vs_log"); plt.show()
```




## Discussion questions — Step 5

**Q1. Does adding more features improve the model?**

**Yes, but with sharply diminishing returns, and the *form* of the model matters more than the *number* of features.** On the identical test set:

| Model | Features | Test $R^2$ | $\Delta R^2$ | Test RMSE |
|---|---|---|---|---|
| 1. Baseline (Step 3) | 4 | 0.7695 | — | $42,044 |
| 2. + `TotalBsmtSF`, `LotArea` | 6 | 0.7957 | **+0.0262** | $39,584 |
| 3. Standardised | 6 | 0.7957 | +0.0262 | $39,584 |
| 4. + `GrLivArea × OverallQual` | 7 | 0.8239 | **+0.0544** | $36,752 |
| 5. `log(SalePrice)`, 6 features | 6 | **0.8642** | **+0.0947** | **$32,269** |

Reading the table:

- **Two extra features buy +2.6 points of $R^2$** and cut RMSE by $2,460. `TotalBsmtSF` earns its place (standardised effect ≈ $8,576 per SD); `LotArea` is the weakest of the six (≈ $6,970 per SD, simple r = 0.26 — in Ames the land is cheap relative to the structure on it).
- **The single interaction term adds more than the two extra features did** (+0.028 on top of model 2), confirming the convexity seen in Step 2: a square foot in a quality-10 house is worth far more than a square foot in a quality-4 house. The fitted marginal value of a square foot runs from ≈ $28/ft² at quality 4 to ≈ $68/ft² at quality 10.
- **Fixing the target distribution beats adding features.** Simply regressing `log(SalePrice)` on the same six predictors gives the largest gain of all ($R^2$ = 0.8642, RMSE $32,269, MAE $20,303) because it matches the multiplicative reality of house prices and tames the heteroscedasticity diagnosed in Step 2. It also improves the weak spots found in Step 4: MAPE on the cheapest quartile falls from 25.1% to 19.6%, overall MAPE from 16.2% to 12.3%, and the top-quartile RMSE from $70,286 to $54,807.

- **Standardisation changes nothing about fit** — $R^2$ is identical to six decimal places, as it must be, since a linear rescaling of the predictors is absorbed by the coefficients. What it changes is *readability*: the intercept becomes the mean training price ($181,441.54) and the coefficients become comparable "USD per standard deviation" effects, which rank the drivers **`OverallQual` ($29,879) > `GrLivArea` ($24,150) > `YearBuilt` ($11,181) > `GarageArea` ($9,531) > `TotalBsmtSF` ($8,576) > `LotArea` ($6,970)**. On the raw scale `OverallQual`'s coefficient of 21,856 looks ~470× larger than `GrLivArea`'s 46.07, but that is an artefact of one being a 1–10 scale and the other square feet — on a common footing the two are close, with quality just ahead.
- **$R^2$ must not be the only criterion.** $R^2$ never falls when a feature is added, so "more is better" is guaranteed on the training set and must be checked on held-out data (as above). Here the test score rises too, so the gains are real rather than overfitting — with 1,168 rows and 7 parameters there is plenty of data per parameter.

**Q2. Are there any signs of multicollinearity in your model?**

**Mild and harmless in the feature-only models; severe — but artificial — once the raw interaction term is added.**

| Feature | Baseline (4) | Extended (6) | + raw interaction | + centred interaction |
|---|---|---|---|---|
| `GrLivArea` | 1.74 | 1.86 | **15.98** | 1.97 |
| `OverallQual` | 2.40 | 2.52 | **7.16** | 2.53 |
| `YearBuilt` | 1.70 | 1.72 | 1.73 | 1.73 |
| `GarageArea` | 1.66 | 1.74 | 1.74 | 1.74 |
| `TotalBsmtSF` | — | 1.63 | 1.67 | 1.67 |
| `LotArea` | — | 1.13 | 1.13 | 1.13 |
| interaction | — | — | **30.59** | 1.19 |

- **Models 1–3 are clean.** Every VIF is below 2.6, far under the usual thresholds (>5 a concern, >10 severe), and no pairwise correlation among the six predictors exceeds **0.593** ( `GrLivArea` – `OverallQual` ). The coefficients are therefore stable and safe to interpret. There *is* visible redundancy — `OverallQual` correlates 0.57 with `YearBuilt` and 0.56 with `GarageArea`, because newer houses are built to higher standards and with bigger garages — and it is exactly this overlap that shrinks `GrLivArea`'s slope from $102 to $53/ft² between the simple and multiple regressions. But it is not severe enough to destabilise the fit.
- **Model 4 shows textbook structural multicollinearity.** `GrLivArea × OverallQual` is by construction almost a linear function of its parents, so its VIF hits **30.6** and `GrLivArea`'s jumps to **16.0**. The symptom is visible in the coefficients: `GrLivArea` collapses from $46.07 to $1.26/ft² and `OverallQual` from $21,856 to $12,059 — the parent terms have become uninterpretable on their own even though the model as a whole predicts *better*.

- **The fix is centring, and it is free.** Multiplying the *mean-centred* parents instead gives the identical test $R^2$ of **0.8239** while every VIF drops back below 2.6 (interaction: 1.19). This is the key lesson about multicollinearity: it inflates the *standard errors and instability of individual coefficients*, not the model's predictive accuracy. If the goal is interpretation, centre the interaction (or drop correlated duplicates); if the goal is pure prediction, the high VIF in model 4 is tolerable — though a regularised model (Ridge/Lasso) would be the more principled answer.

---

# Report — Strengths, Limitations and Conclusions

## Summary of what was done

| Step | Decision taken |
|---|---|
| Preprocessing | 19 columns with `NA` repaired by four documented rules ( `"None"` for absent-feature categoricals, `0` for their numeric counterparts, `Neighborhood` median for `LotFrontage` , mode for `Electrical` ); all 1,460 rows retained; no imputation touched the target |
| Features | `GrLivArea` , `OverallQual` , `YearBuilt` , `GarageArea` ; extended with `TotalBsmtSF` , `LotArea` , and an interaction term |
| Validation | one 80/20 split ( `random_state = 42` ) reused for every model, plus 5-fold CV on the baseline |
| Best model | `log(SalePrice)` on six features: **test $R^2$ = 0.8642, RMSE = $32,269, MAE = $20,303** |

## Strengths

1. **Strong explanatory power for four variables.** $R^2$ = 0.77 from `GrLivArea` , `OverallQual` , `YearBuilt` and `GarageArea` alone — 4 of 79 available predictors — showing how concentrated the price signal is in size and quality.
2. **Fully interpretable.** Every coefficient is a price in dollars ($52.97 per ft², $22,708.55 per quality point). A surveyor can audit the model by hand, which no ensemble method allows; the worked example above reproduces `model.predict()` with a pen-and-paper sum.
3. **No overfitting, and cheap to fit.** Test $R^2$ ≥ train $R^2$, 5 parameters, millisecond training, no hyperparameters to tune. The model is a defensible baseline that any more complex approach must beat.
4. **Honest, reproducible pipeline.** A fixed `random_state` , one split shared by all five specifications, metrics always reported on held-out data, and imputation that never looks at `SalePrice` .
5. **Diagnostics drove the improvements.** The non-linearity found in Step 2 was not left as a remark — it motivated the interaction term and the log target, which together

lifted $R^2$ from 0.77 to 0.86 and cut typical error by $9,775.

## Limitations

1. **Linearity is the wrong shape for this market.** Price is multiplicative (a quality upgrade is worth proportionally more on a large house), and the quality ladder is convex: +$28k from quality 5→6 versus +$93k from 8→9. A single slope must average these, which is precisely why the interaction and log specifications do better.
2. **Heteroscedastic, skewed residuals.** The residual spread grows with the prediction and skews +2.94. OLS's constant-variance assumption is violated, so the usual confidence intervals and p-values on the coefficients would be understated. Any interval reported from this model should be treated as indicative only.
3. **Systematic failure at the top of the market.** The top price quartile is under-predicted by $29,665 on average with an RMSE of $70,286 (2.6× the rest), and the single worst miss is $308,144 on a $755,000 home. Luxury value lives in features the model never sees.
4. **Relative error is worst at the bottom.** 25.1% MAPE in the cheapest quartile: four positive coefficients give the model no mechanism to predict a genuinely low price, so it pulls the cheapest houses up toward the mean.
5. **No categorical information at all.** `Neighborhood`, `KitchenQual`, `ExterQual`, `SaleCondition` and 39 other text columns are dropped. In real estate, location is the dominant price factor — omitting it is the model's biggest single gap, and it means the coefficients absorb location effects they should not (omitted-variable bias).
6. **Outliers are left in.** The two `SaleCondition = Partial` houses (`Id` 524 and 1299: 4,676 and 5,642 ft², quality 10, yet sold for $184,750 and $160,000) are high-leverage points that flatten the `GrLivArea` slope for everyone else. They are incomplete sales, not ordinary market transactions.
7. **Narrow domain of validity.** One county (Ames, Iowa), one period (2006–2010, spanning the housing crash), and no inflation adjustment. The coefficients should not be transported to another market or to today's prices.
8. **Single split.** The 5-fold CV range ($R^2$ 0.53–0.80) shows how much the score depends on which luxury houses land in the test set; the headline 0.7695 is one draw from a fairly wide distribution.

## What would improve it next

**One-hot encode `Neighborhood` and the quality ordinals** (likely the single largest gain available), **keep the log target**, **drop or cap the `Partial` sales**, **use Ridge or Lasso** so correlated features and interactions can be added without destabilising the coefficients, and **report k-fold CV rather than a single split**. A gradient-boosted tree on the full 79 columns is the natural next benchmark — but it would give up the clean dollar-per-square-foot interpretation that makes this model worth keeping as a reference.

# Consolidated answers to all discussion questions

| Step | Question | Answer in brief |
|---|---|---|
| 1 | Columns with missing values? | **19 of 81** (7,829 cells, 6.62%); 5 columns >50% missing |
| 1 | Strategy and why? | Category-aware imputation, not deletion — `NA` is a documented "feature absent" level; `dropna()` would leave **0 rows** |
| 2 | Strongest correlation? | `OverallQual` , **r = +0.791** (positive); then `GrLivArea` 0.709, `GarageArea` 0.623, `YearBuilt` 0.523 — all positive |
| 2 | Non-linearity? | Yes — Spearman > Pearson for all four ( `YearBuilt` +0.130), a convex quality ladder, and variance growing 4.6× with size |
| 3 | Regression equation? | `SalePrice = −845,603.74 + 52.97·GrLivArea + 22,708.55·OverallQual + 395.57·YearBuilt + 57.70·GarageArea` |
| 3 | Effect of one unit of `GrLivArea` ? | **+$52.97 per ft²** (+$5,297 per 100 ft²; +$27,778 per SD), down from $102/ft² in the simple regression |
| 3 | Negative coefficients? | **None** — all four are positive; a negative one would signal a true negative effect, collinearity suppression, or mis-specified form |
| 4 | $R^2$ and what it indicates? | **0.7695** test (train 0.7454; CV 0.7362 ± 0.1027) — 77% of price variance explained, no overfitting, typical miss $26,879 |
| 4 | Where does it perform poorly? | **High end in dollars** (Q4: −$29,665 bias, RMSE $70,286) and **low end in percentage** (Q1: 25.1% MAPE); reliable in the $127k–209k middle |
| 5 | Do more features help? | Yes — $R^2$ 0.7695 → 0.7957 (+2 features) → 0.8239 (+interaction) → **0.8642** (log target), with diminishing returns |
| 5 | Multicollinearity? | Mild in models 1–3 (all VIF < 2.6, max pairwise r = 0.593); severe but artificial with the raw interaction (VIF 30.6), fixed by centring at identical $R^2$ |

In [34]:

```
# ---- Final reproducibility stamp ----------------------------------------
print("Figures written to", FIGDIR.resolve())
for f in sorted(FIGDIR.glob("*.png")):
    print("  ", f.name)
print(f"\nHeadline results")
print(f"  Baseline (4 features)  : test R² = {r2:.4f}, RMSE = {rmse:,.0f}
USD")
print(f"  Best model (log target): test R² = {r2_score(y_test,
pred_log):.4f}, "
      f"RMSE = {np.sqrt(mean_squared_error(y_test, pred_log)):,.0f} USD")
```

```
Figures written to /Users/lethiminhngoc/Downloads/house price
prediction/figures
   01_missing_values.png
   02_scatter_features.png
   03_correlations.png
   04_nonlinearity.png
   05_actual_vs_predicted.png
   06_residuals.png
   07_error_by_price_band.png
   08_standardised_coefficients.png
   09_correlation_heatmap.png
   10_model_comparison.png
   11_baseline_vs_log.png

Headline results
  Baseline (4 features)  : test R² = 0.7695, RMSE = 42,044 USD
  Best model (log target): test R² = 0.8642, RMSE = 32,269 USD
```